Introduction to Type Systems

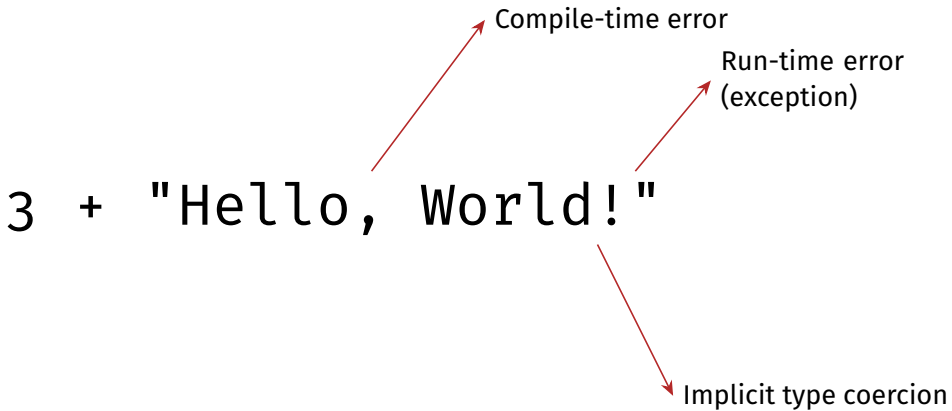
Zoe Paraskevopoulou

School of Electrical and Computer Engineering
National Technical University of Athens

2025



```
3 + "Hello, World!"
```


Compile-time error
Run-time error
(exception)
3 + "Hello, World!"
Implicit type coercion

```c
#include <stdio.h>

int main() {
   char* s = 3 + "Hello, World!";
   printf("%s\n", s);
}
```

```c
#include <stdio.h>

int main() {
   char* s = 3 + "Hello, World!";
   printf("%s\n", s);
}
```

```
lo, World!
```

```
3 + 'a'
```

```c
#include <stdio.h>

int main() {
   printf("%d", 3 + 'a');
   return 0;
}
```

```c
#include <stdio.h>

int main() {
   printf("%f", 3 + 'a');
   return 0;
}
```

```c
#include <stdio.h>

int main() {
   printf("%c", 3 + 'a');
   return 0;
}
```

```c
#include <stdio.h>

int main() {
   printf("%s", 3 + 'a');
   return 0;
}
```

```c
#include <stdio.h>

int main() {
   printf("%d", 3 + 'a');
   return 0;
}
```

```
100
```

```c
#include <stdio.h>

int main() {
   printf("%f", 3 + 'a');
   return 0;
}
```

```
0.000000
```

```c
#include <stdio.h>

int main() {
   printf("%c", 3 + 'a');
   return 0;
}
```

```
d
```

```c
#include <stdio.h>

int main() {
   printf("%s", 3 + 'a');
   return 0;
}
```

```
[1] 39911
segmentation fault
```

Language Safety

Disclaimer: no clear-cut definition

- **“A language is safe if it cannot violate its own abstractions”**
  - Data abstraction
  - Control abstraction
- **Unsafe** operations/constructs
  - Out-of-bounds array access
  - Down-casting
  - Integer overflow
  - …
- **Trapped** vs **untrapped** errors

Exception when execution **goes wrong** (safe)

Undefined behavior. Execution **continues** after **erroneous** operation (unsafe)

heap corruption, stack corruption, code injection, …

A Guide to Undefined Behavior in C and C++, Part 1. John Regehr.

Type Safety

- **“The operations that can be applied to data are only those defined for its type”**
- Type checking: **static** or **dynamic**

	Static typing	**Dynamic typing**
Strong typing	ML, Java, Rust, Haskell, Go, …	LISP, Python, Ruby, …
Loose typing	C, C++	JavaScript

Static Type System

> *A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute.*
>
> **Types and Programming Languages**, Benjamin C. Pierce

- **Static checking** that the language expressions behave as their specification (type) dictates
- **Rejection** of programs that lead to **errors**
- **Conservative** approach (it is possible to reject programs that do not actually lead to errors)
- **Static** safety guarantees:
  - No **untrapped** errors arise (safety)
  - No **trapped errors that the type system is designed to prevent** arise
  - Other **trapped errors** may still occur. It is the programmer's responsibility to avoid them (e.g., access out of bounds errors, division by zero errors, ...)

Static Type System

- Advantages
  - Documentation (machine checked)
  - Abstraction
    - **“Type structure is a syntactic discipline for enforcing levels of abstraction.”** Reynolds (1983)
  - Efficiency
    - **Dynamic checks are avoided** (zero-cost abstractions)
    - Type-specific optimizations

Escape Mechanisms

Safety is a spectrum

- Most languages with strong static types provide escape mechanisms
- **Haskell**: unsafePerformIO
- **OCaml**: Obj.magic
- **Rust**: unsafe
- …

Expressiveness of Type Systems

- A type system can describe a **wide range of properties** of a program
- Examples:
  - Security (e.g., information-flow control)
  - Complexity
  - Resource usage (linear and affine type systems)
  - Refinement types
  - Dependent types
  - …
- Greater expressiveness = increased difficulty in usage and checking

Type Theory

- Formal study of type systems
- Branch of mathematics, logic, and philosophy
- Foundation of mathematics
- Created to avoid paradoxes in the then-existing foundation of mathematics
- Curry–Howard isomorphism
  - **Syntactic** analogy between **type systems** (pure) and **formal proofs**

Type Systems in This Course

- Simply typed lambda calculus + extensions
- System F
- Hindley–Milner type system (typed lambda calculus with parametric polymorphism)
- References
- (A taste of) dependent types

Simply Typed Lambda Calculus

$t : A$ The term t has type A

If the term t has free variables

$$\Gamma \vdash t : A$$

$$\Gamma = x_1 : A_1, \ldots, x_n : A_n$$

Simply Typed Lambda Calculus

Terms

$$t, u \ ::= \ \lambda(x : A).\, t \ \mid \ t\ u \ \mid \ x$$

Types

$$A, B \ ::= \ \alpha \ \mid \ A \to B$$

Simply Typed Lambda Calculus

Terms

$$t, u \ ::= \ \lambda(x : A).\, t \ \mid \ t\ u \ \mid \ x$$

Type annotation (Church style)

Types

$$A, B \ ::= \ \alpha \ \mid \ A \to B$$

Simply Typed Lambda Calculus

Terms

$$t, u \ ::= \ \lambda(x : A).\, t \ \mid \ t\ u \ \mid \ x$$

Types

$$A, B \ ::= \ \alpha \ \mid \ A \to B$$

Type variable representing an uninterpreted base type

Typing Rules

$$\frac{\Gamma(x) = A}{\Gamma \vdash x : A}\ \textsc{Var}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x : A).\, t : A \to B}\ \textsc{Abs} \qquad \frac{\Gamma \vdash t : A \to B \qquad \Gamma \vdash u : A}{\Gamma \vdash t\ u : B}\ \textsc{App}$$

Examples

$\lambda(f : \alpha \to \beta)(x : \alpha).\, f\ x$

Typing derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{f : \alpha \to \beta,\, x : \alpha \vdash f : \alpha \to \beta}\ \textsc{Var} \quad \dfrac{}{f : \alpha \to \beta,\, x : \alpha \vdash x : \alpha}\ \textsc{Var}}{f : \alpha \to \beta,\, x : \alpha \vdash f\ x : \beta}\ \textsc{App}}{f : \alpha \to \beta \vdash \lambda(x : \alpha).\, f\ x : \alpha \to \beta}\ \textsc{Abs}}{\vdash \lambda(f : \alpha \to \beta)(x : \alpha).\, f\ x : (\alpha \to \beta) \to \alpha \to \beta}\ \textsc{Abs}$$

Examples

$$\lambda(f : \beta)(x : \alpha).\, f\ x$$

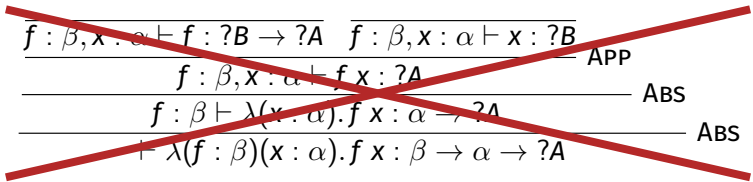
X

$$\dfrac{\dfrac{\dfrac{\overline{f : \beta,\, x : \alpha \vdash f : ?B \to ?A} \qquad \overline{f : \beta,\, x : \alpha \vdash x : ?B}}{f : \beta,\, x : \alpha \vdash f\ x : ?A}\ \textsc{App}}{f : \beta \vdash \lambda(x : \alpha).\, f\ x : \alpha \to ?A}\ \textsc{Abs}}{\vdash \lambda(f : \beta)(x : \alpha).\, f\ x : \beta \to \alpha \to ?A}\ \textsc{Abs}$$

Examples

$$\lambda(f : \beta)(x : \alpha).\, f\ x$$

The term is not well-typed



Type Safety

- **“Well-typed programs do not get stuck.”**
- **Preservation**

$$\text{If} \quad \vdash t : A \quad \text{and} \quad t \to t' \quad \text{then} \quad \vdash t' : A$$

- **Progress**

$$\text{If} \quad \vdash t : A \quad \text{then} \quad \exists\, t',\, t \to t' \quad \text{or} \quad \text{value}(t) = \text{true}$$

Normalization

$$\text{If} \quad \vdash t : A \quad \text{then } t \text{ is normalizable}$$

- Tait (1967) using **logical relations**
- Full beta reduction: **strong normalization** vs **weak normalization**
- Weak normalization

$$\text{If} \quad \vdash t : A \quad \text{then} \quad \exists v, \;\; t \to^* v \;\; \text{and} \;\; \text{value}(v) = \text{true}$$

Type Reconstruction

- **Type inference** for STLC is **decidable** (Hindley)
- Constraint **generation** and **solving** (first-order unification) algorithm
- Type annotations may be omitted (implicit types)

Simple Extensions

- Product types
- Sum types
- Base types: unit, nat, bool

In Rocq…

Other Simple Extensions

- Tuples and Records (generalization of products)
- Variants (generalization of sums)
- Lists (can be derived using variants and recursive types)
- General recursion